SESSION 10

Cloud Training Jobs & Experiment Tracking

Azure ML SDK v2 | MLflow | Sweep Jobs | Model Registry

Day 2 | 11:00 AM – 12:00 PM | 60 minutes

OBJECTIVES

What You'll Master Today

01 Submit Cloud Training Jobs

Run scripts on Azure ML Compute Clusters using SDK v2

02 Track with MLflow

Log params, metrics & artifacts — autolog() or manual

03 Cloud-Scale HPO

Sweep Jobs: Bayesian sampling + Bandit early termination

04 Compare Runs in Studio

Select runs → Compare → parallel coordinates chart

05 Register Best Model

Promote model: experiment → staging → production

CORE CONCEPTS

Azure ML Job Types

Command Job

Single-script training run

- Simple & fast to configure
- Wrap any Python script
- Submit from SDK or YAML

Sweep Job

Hyperparameter optimization

- 30+ parallel trials
- Bayesian / random sampling
- Bandit early termination

Pipeline Job

Multi-step ML workflow

- DAG of reusable components
- Data prep → train → evaluate
- Cacheable steps

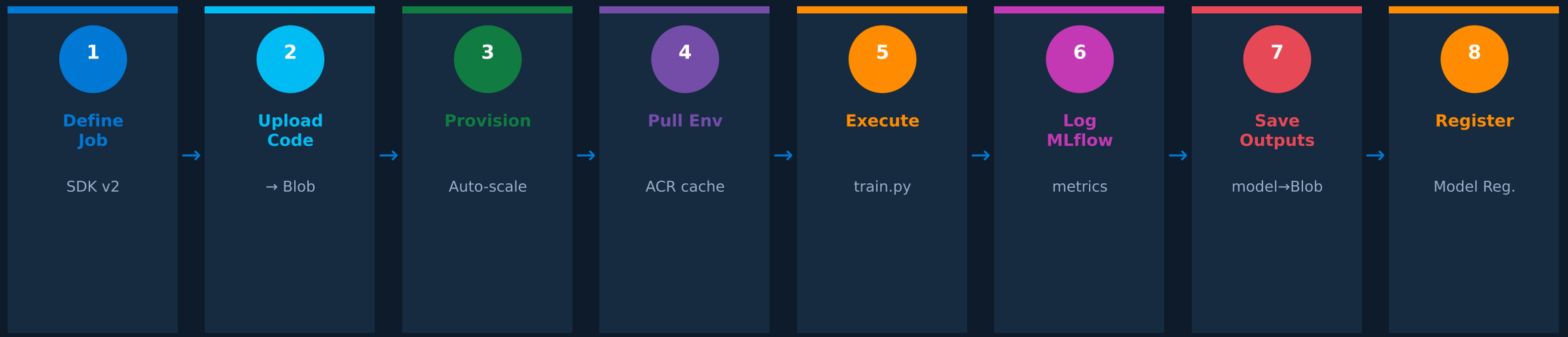
AutoML Job

Automated model selection

- No-code experimentation
- Azure chooses algorithm
- Benchmark quickly

JOB LIFECYCLE

From SDK to Cloud Compute — 8-Stage Lifecycle



COMMAND JOB

Submitting a Training Job — SDK v2

```python
from azure.ai.ml import MLClient, command, Input
from azure.identity import DefaultAzureCredential

ml_client = MLClient(
    DefaultAzureCredential(),
    subscription_id, resource_group, workspace
)

job = command(
    code='./scripts',
    command='python train.py --lr 0.05',
    environment='azureml:credit-model-env:1',
    compute='cpu-cluster',
    experiment_name='credit-default-model',
    tags={'framework': 'sklearn'}
)

returned_job = ml_client.jobs.create_or_update(job)
print('Studio URL:', returned_job.studio_url)
```

Key Parameters

compute

Target cluster
(auto-scales 0 → N nodes)

environment

Versioned Docker env
(cached in ACR)

code

Local folder snapshot
uploaded to Blob

experiment

Groups related runs
in Studio dashboard

MLFLOW TRACKING

Experiment Tracking — Auto vs. Manual

mlflow.autolog()

One line captures everything

```
import mlflow
mlflow.autolog()  # done!
```

Auto-captures:

- All __init__ parameters
- Training score
- Feature importances
- Fitted model artifact

Works with: sklearn, XGBoost, PyTorch, TensorFlow

Best for: Quick experiments

Manual mlflow.log_*()

Fine-grained control per metric

```
with mlflow.start_run():
  mlflow.log_param('lr', lr)
  mlflow.log_metric(
      'val_auc', val_auc,
      step=epoch)
  mlflow.log_figure(
      fig, 'confusion.png')
  mlflow.sklearn.log_model(
      model, 'model')
```

Best for: Production pipelines

SWEEP JOBS

Cloud-Scale Hyperparameter Optimization

```python
from azure.ai.ml.sweep import Choice, Uniform, BanditPolicy

sweep_job = command_job.sweep(
    sampling_algorithm='bayesian',
    primary_metric='val_auc',
    goal='Maximize',
    search_space={
        'learning_rate': Uniform(0.001, 0.1),
        'max_depth':     Choice([3, 5, 7, 10]),
        'n_estimators':  Choice([100, 200, 300, 500])
    },
    limits={
        'max_total_trials': 30,
        'max_concurrent_trials': 5,
    },
    early_termination_policy=BanditPolicy(
        slack_factor=0.1,
        evaluation_interval=1
    )
)
```

Sampling Algorithms

Random

No learning between trials

Baseline

Grid

All combos; exhaustive

Small spaces

Bayesian

Learns from previous trials

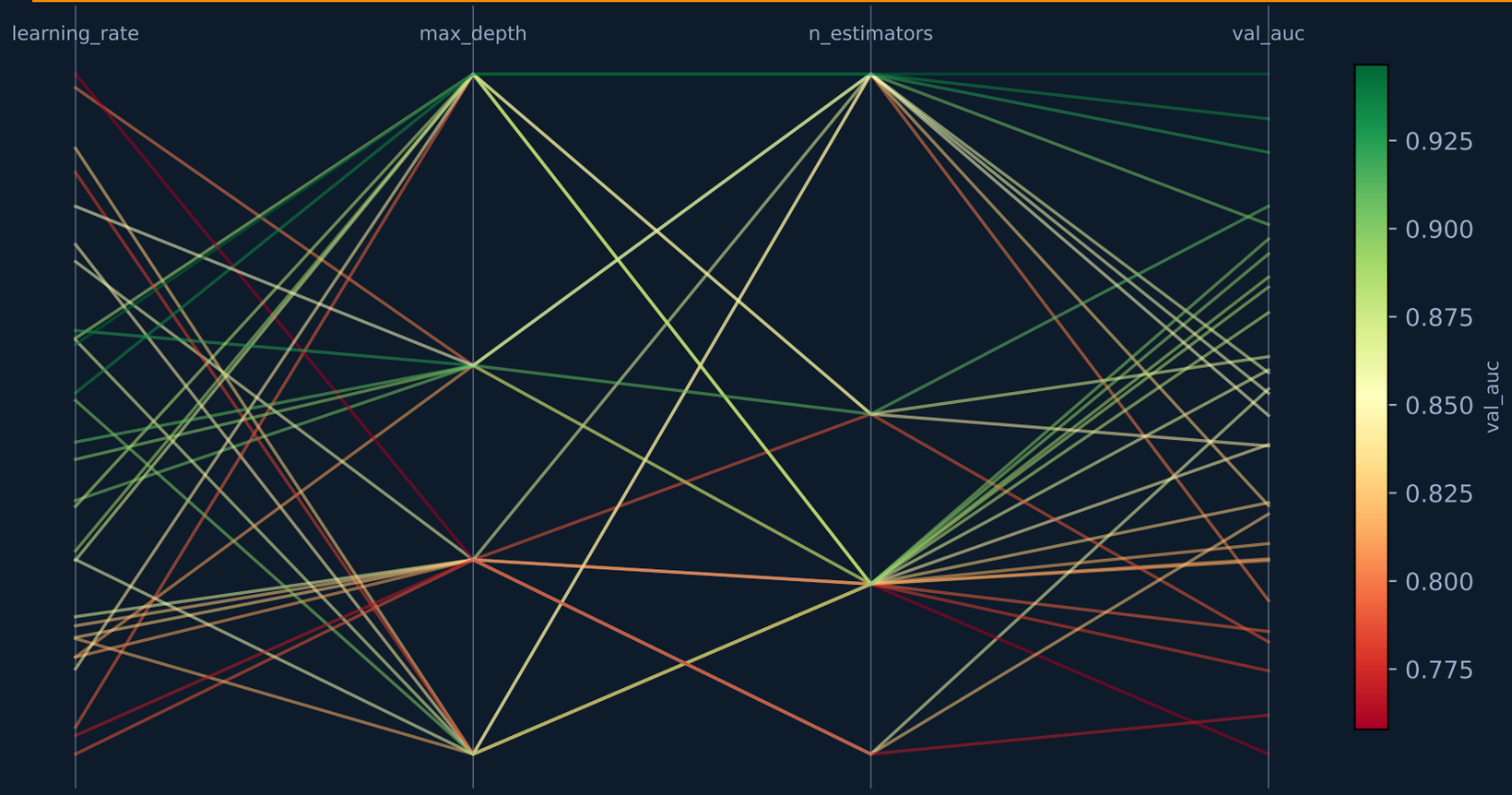
Best: 5x faster

Bandit Policy — Cost Control

Kills trials >10% behind the current best.
Saves up to 60% compute vs. running all trials.

SWEEP VISUALISATION

Reading the Parallel Coordinates Chart



How to Read It

Green lines = high val_auc
(good hyperparameter combos)

Red lines = poor trials
(terminated early by Bandit)

Steep angles = strong
param-metric correlations

Cluster of green lines =
optimal search region

ENVIRONMENTS

Reproducible Environments — Docker + Conda

```
env = Environment(
    name='credit-model-env',
    version='1',
    conda_file='environment.yml',
    image='mcr.microsoft.com/azureml/
        openmpi4.1.0-ubuntu20.04:latest'
)
ml_client.environments.create_or_update(env)
```

```
# environment.yml
name: credit-model-env
dependencies:
  - python=3.10
  - pip:
    - scikit-learn==1.3.0
    - mlflow==2.7.0
    - pandas==2.0.3
```

Environment Lifecycle

1. Define

Python SDK or conda YAML

2. Register

Stored in workspace registry

3. Build

Docker image built in ACR (~10 min first time)

4. Cache

Subsequent jobs pull from ACR (<1 min)

5. Audit

Every model links to exact env version

STUDIO NAVIGATION

Comparing Runs in Azure ML Studio

Azure ML Studio → **Jobs** → **credit-default-model** → **Runs** → **Compare**

01 Navigate to Jobs

Studio home → Jobs (left sidebar)
Select your experiment

02 Select Runs

Ctrl+Click 3+ runs
Sortable by any column

03 Click Compare

Top toolbar → Compare
Side-by-side metrics

04 Parallel Coords

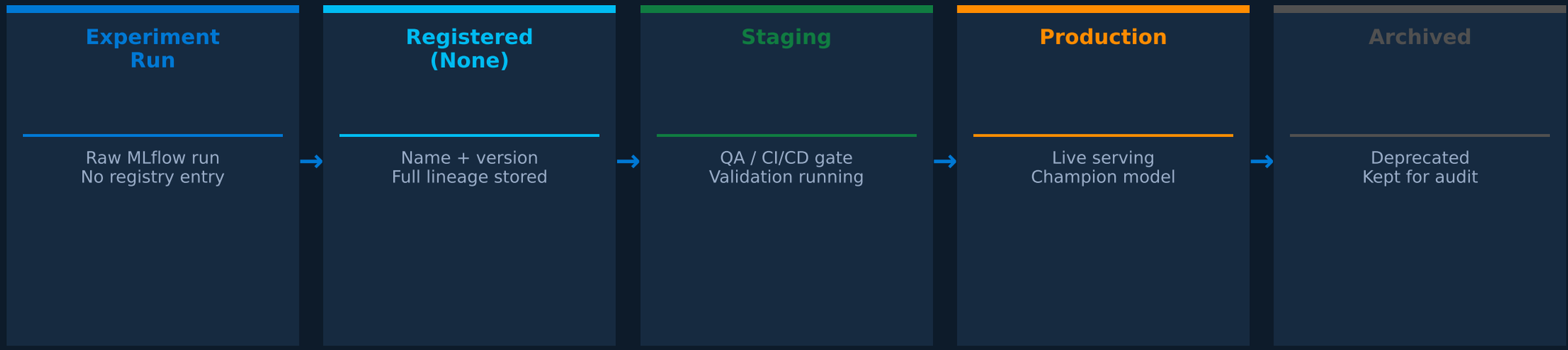
Switch to Parallel Coords tab
Color = val_auc

05 Promote Best

Right-click → Register Model
Directly to Registry

MODEL REGISTRY

Model Registry — Versioning & Promotion




```
model = Model(
    path=f'azureml://jobs/{job_name}/outputs/model',
    name='credit-default-classifier',
    type=AssetTypes.MLFLOW_MODEL,
    tags={'val_auc': '0.923', 'framework': 'sklearn'}
)
registered = ml_client.models.create_or_update(model)
```

STUDIO NAVIGATION

Azure ML Studio — Navigation Map for AI & MLOps

Jobs

- All submitted jobs
- Experiment groups
- Job logs & outputs
- Child runs (Sweep)

Data

- Datastores (connections)
- Data Assets (versioned)
- Upload & preview
- URI path management

Models

- Version history
- Tags & lineage
- Deploy endpoint
- Promote label

Compute

- Compute Instances
- Compute Clusters
- Inference Clusters
- Serverless compute

Environments

- Curated (ready-to-use)
- Custom (yours)
- Build logs (ACR)
- Package details

Endpoints

- Online (real-time API)
- Batch (large datasets)
- Deployment versions
- Traffic splitting

TAKEAWAYS

Key Takeaways

Cloud training removes all local resource limits

Run 30 parallel trials on GPU/CPU clusters while you sleep — zero local overhead

mlflow.autolog() is your fastest start

One line captures all params, metrics, and the fitted model — no boilerplate

Sweep Jobs = Optuna at cloud scale

Bayesian + Bandit termination → converges 5x faster, uses 60% less compute

Environments enforce reproducibility

Versioned Docker + conda spec, cached in ACR — same result 6 months later

Model Registry formalizes your promotion path

experiment → staging → production with full lineage, tags, and audit trail